

República Bolivariana de Venezuela

Ministerio del Poder Popular para la Educación

Universidad Nacional Experimental de Telecomunicaciones e Informática

Programación III

Distrito Capital – Caracas

Ionic Ejercicio SD2

(Link de GitHub: <https://github.com/Kikerivera10/portafolio-kike-ionic/tree/fase2-interactiva>)

Estudiante:

José Rivera C.I 27767299

MSc. Carlos Márquez

Caracas, 07/05/2026

Introducción

La evolución de las aplicaciones híbridas modernas exige la transición de interfaces puramente informáticas hacia ecosistemas dinámicos capaces de responder a estímulos en tiempo real y preservar estados de forma asíncrona. Si bien la entrega anterior consolidó la arquitectura estructural desvinculada mediante Componentes Standalone, esta Fase 2 aborda la optimización funcional del portafolio enfocado en la experiencia de evaluación local.

El desarrollo de esta fase se estructuró bajo dos premisas técnicas esenciales: asegurar que el flujo de datos sea completamente agnóstico a la conectividad de red externa (garantizando el correcto funcionamiento en el entorno local del evaluador) e integrar mecánicas lúdicas de alta reactividad dentro de contenedores globales como ion-menu. A través de la manipulación avanzada del ciclo de vida de Angular, la gestión de eventos del DOM y el uso del subsistema nativo de almacenamiento del navegador, el proyecto se eleva de una plantilla estática a un portafolio interactivo de grado de ingeniería.

Capítulo 1: Animación Reactiva y Gestión del Ciclo de Vida

1.1. Inyección de Comportamiento Dinámico en Skills

En el módulo de Información Personal, la presentación estática de competencias técnicas fue sustituida por un algoritmo de llenado progresivo. Utilizando una estructura tipada controlada por una interfaz explícita de TypeScript (interface Skill), se desacopló el valor real de la competencia de su estado de renderizado actual:

```
interface Skill {  
  nombre: string;  
  nivel: number;           // Cota superior (0.0 a 1.0)  
  progresoActual: number; // Modificado asíncronamente por el temporizador  
}
```

1.2. Orquestación con ngOnInit y Control de Hilos con setInterval

Para garantizar que la animación se ejecute inmediatamente después de que la directiva estructural haya inicializado las propiedades del componente, la lógica se inyectó en el gancho de ciclo de vida ngOnInit. Mediante el uso controlado de temporizadores nativos (setInterval), el motor de detección de cambios de Angular re-renderiza las barras de progreso en incrementos estocásticos de 0.05 cada 50 milisegundos hasta interceptar el límite superior definido por nivel.

Este enfoque mitiga el bloqueo del hilo principal de ejecución (Main Thread), ofreciendo una experiencia visual fluida que, como sostiene Freeman (2022), resulta crítica para retener la atención en interfaces web de alto rendimiento.

Capítulo 2: Robustez en el Flujo de Datos y Persistencia Local

2.1. Mitigación de Fallos de Protocolo mediante Bandeja de Salida Local

El módulo de contacto original dependía exclusivamente de la invocación del protocolo del sistema operativo mailto: . En un entorno de evaluación estrictamente local, si el host carece de un cliente de correo nativo configurado (ej. Outlook o Apple Mail), dicha acción resulta en un evento silencioso sin retroalimentación.

Para resolver este vector de fallo, se implementó un patrón de Bandeja de Salida Simulada que actúa en paralelo al enlace externo.

2.2. Implementación de LocalStorage y Serialización JSON

Cada vez que el formulario de contacto intercepta un evento de envío válido, los datos capturados a través del enlace bidireccional (`[[ngModel]]`) se empaquetan en un objeto estructurado que incluye una marca de tiempo generada dinámicamente (`new Date().toLocaleString('es-VE')`).

El estado asíncrono se persiste localmente invocando el API de almacenamiento del navegador:

```
this.historialMensajes.unshift(nuevoMensaje);
```

```
localStorage.setItem('kike_portfolio_mensajes', JSON.stringify(this.historialMensajes));
```

La des-serialización de cadenas JSON se realiza de forma segura durante la inicialización del componente, volcando los datos directamente sobre una directiva estructural `*ngFor` en la capa de presentación. Así, el evaluador puede rellenar campos, enviar mensajes simulados y verificar el flujo de persistencia de datos de manera inmediata sin salir de la vista de la aplicación.

Capítulo 3: Componentes Lúdicos Empotrados en la Interfaz (Easter Eggs)

3.1. Arquitectura y Lógica Estocástica de "NovaClicker"

Para demostrar el dominio sobre la manipulación fina del árbol del DOM y eventos asíncronos concurrentes, se integró un minijuego interactivo tipo Clicker ("NovaClicker") empotrado en la base del menú lateral global (AppComponent).

El motor de juego opera bajo un ciclo de vida restringido de 10 segundos controlado por intervalos. La posición del objetivo interactivo se calcula dinámicamente mediante funciones de dispersión aleatoria aplicadas sobre coordenadas relativas CSS aseguradas:

```
moverBoton() {  
  const randomLeft = Math.floor(Math.random() * 65) + 5;  
  const randomTop = Math.floor(Math.random() * 40) + 10;  
  this.botonPosicionLeft = `${randomLeft}%`;  
  this.botonPosicionTop = `${randomTop}px`;  
}
```

3.2. Gestión de Burbujeo de Eventos en Menús Flexibles

Un desafío técnico crítico al insertar elementos altamente interactivos dentro de un componente de navegación jerárquico como ion-menu es el fenómeno de Burbujeo de Eventos (Event Bubbling). Al hacer clics sucesivos y rápidos sobre el objetivo móvil, el evento de pulsación puede propagarse hacia los nodos padres contenedores, provocando un colapso accidental del panel lateral o transiciones de ruta no deseadas.

La anomalía se resolvió interceptando explícitamente el objeto de evento nativo y deteniendo su propagación en el árbol de ejecución:

```
registrarClic(event: Event) {  
  event.stopPropagation(); // Detiene el flujo ascendente del evento en el DOM  
  if (!this.juegoActivo) return;  
  this.puntuacion++;  
  this.moverBoton();  
}
```

Adicionalmente, el récord máximo alcanzado por el usuario se enlaza al almacén de persistencia local del sistema bajo la clave `kike_game_record`, permitiendo que el estado competitivo del juego sobreviva a recargas completas de la página (F5).

Capítulo 4: Gestión de Paquetes y Refactorización del Bundle

4.1. Limpieza de Declaraciones y Advertencias de Compilación

Durante la auditoría de rendimiento de la compilación local (Application bundle generation), el compilador nativo de Angular (angular-compiler) arrojó la advertencia de optimización NG8113, identificando que el componente `@ionic/angular/standalone` denominado `IonBadge` había sido declarado e importado formalmente en el decorador `@Component` de `AppComponent` pero no poseía selectores activos en la plantilla HTML correspondiente.

Siguiendo los estándares internacionales de arquitectura de software limpio definidos por Martin (2008), se procedió a realizar una refactorización estricta. Al remover la dependencia inactiva tanto del árbol de importaciones del módulo TypeScript como del arreglo de inyección de metadatos, se redujo el overhead del analizador sintáctico. El resultado final consolidó un proceso de reconstrucción (Rebuild) impecable, arrojando cero advertencias y optimizando el tamaño del chunk inicial a solo 23.11 kB, acelerando drásticamente el tiempo de respuesta inicial.

Conclusión

La culminación de esta Fase 2 demuestra que el desarrollo de software móvil e híbrido va mucho más allá de estructurar interfaces estéticas; radica en la capacidad de construir sistemas altamente reactivos, predecibles frente a fallos y eficientes en el consumo de recursos de computación locales.

Haber implementado lógica de animación asíncrona, técnicas avanzadas de aislamiento de eventos del DOM y persistencia nativa con LocalStorage no solo robustece la aplicación ante contingencias de conectividad de la máquina local del evaluador, sino que evidencia un criterio técnico maduro alineado con los pilares de la Ingeniería Informática. Este portafolio interactivo se consolida, por lo tanto, como un sistema de software integral, escalable y metodológicamente documentado, listo para su defensa académica.

Referencias Bibliográficas

Literatura de Ingeniería y Diseño

- Norman, D. (2013). The Design of Everyday Things. Basic Books. (Principios de retroalimentación inmediata aplicados a los componentes de alerta visual y bandejas de interacción local).
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. (Estrategias de refactorización utilizadas para la eliminación de código muerto e imports redundantes).
- Norman, D. (2013). The Design of Everyday Things. Basic Books. (Principios de retroalimentación inmediata aplicados a los componentes de alerta visual y bandejas de interacción local).

Documentación Técnica Digital:

- Angular Team (2026). Angular Guide: Standalone Components and Reactive State Management. Recuperado de angular.dev
- Ionic Framework (2026). Ionic Standalone UI Components Reference Manual. Recuperado de ionicframework.com/docs

Link de GitHub: <https://github.com/Kikerivera10/portafolio-kike-ionic/tree/fase2-interactiva>